

Algorytmy Zaawansowane

Opracował Radosław Głasek na podstawie wykładów Michała Tuczyńskiego

Spis treści

1	Wykład 1	2
1.1	Notacja asymptotyczna — przypomnienie	2
1.2	Strategia typu <i>Dziel i Zwyciężaj</i> (<i>Divide and Conquer</i>)	3

Wykład 1

Notacja asymptotyczna — przypomnienie

$$f, g : \mathbb{N} \rightarrow \mathbb{R}_+$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} g(n) \leq c \cdot f(n)$$

$$g(n) = \Omega(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c \cdot f(n) \leq g(n)$$

$$g(n) = \Theta(f(n)) \Leftrightarrow \exists_{c_1, c_2 \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow f(n) = \Omega(g(n))$$

$$g(n) = \Theta(f(n)) \Leftrightarrow (g(n) = \mathcal{O}(f(n)) \wedge g(n) = \Omega(f(n)))$$

$$n^a = \mathcal{O}(n^b), \quad 0 < a \leq b$$

$$\log n = \mathcal{O}(n^\epsilon), \quad \epsilon > 0$$

$$a^n = \mathcal{O}(b^n), \quad 1 < a \leq b$$

$$\log_a n = \Theta(\log_b n), \quad a, b > 1$$

$$n^a = \mathcal{O}(b^n), \quad 0 < a, \quad b > 1$$

$$T(n) = 5n^3 + 2n^2 + 100n \log n \underset{n \rightarrow \infty}{\approx} 5n^3 = \Theta(n^3)$$

Procedura rekurencyjna

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & n \geq 1 \end{cases}$$

Oczywiste jest, że $n! = \prod_{i=1}^n i$

Ciąg Fibonacciego F_n - ciąg Fibonacciego.

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n \geq 2 \end{cases}$$

Algorytm 1: Rekurencyjna konstrukcja ciągu Fibonacciego

```

function FIBB( $n$ )
1:   if  $n = 0$  then
2:     return 0
3:   if  $n = 1$  then
4:     return 1
5:   return FIBB( $n - 1$ ) + FIBB( $n - 2$ )
    
```

Wywołanie rekurencyjne dla (typowo łatwiejszych i mniejszych) instancji tego samego problemu.

Strategia typu *Dziel i Zwyciężaj* (*Divide and Conquer*)

Algorytm typu *dziel i zwyciężaj* składa się z trzech części

1. *Dziel* – problem jest dzielony na mniejsze podproblemy
2. *Zwyciężaj* – podproblemy są rozwiązywane rekurencyjnie
3. *Połącz* – rozwiązanie całego problemu jest konstruowane z rozwiązań podproblemów

Zakładamy, że (zazwyczaj) w fazie *dziel* problem jest dzielony na co najmniej dwa podproblemy i podproblemy są rozłączne.

Przykład 1.1. Mnożenie liczb n -cyfrowych x, y – liczby naturalne n -cyfrowe.

Aby policzyć $x \cdot y$ można użyć mnożenia pisemnego (jak na kartce). Tym sposobem musimy wykonać $\mathcal{O}(n^2)$ mnożeń cyfr. Spróbujemy znaleźć lepszą metodę. Dzielimy x i y na „dwie połowy”. Zakładamy, dla uproszczenia, że n jest parzyste.

$$x = x_1 x_2 \cdots x_n = \underbrace{x_1 x_2 \cdots x_{n/2}}_{x_L} \underbrace{x_{n/2+1} \cdots x_n}_{x_R} = 10^{n/2} x_L + x_R$$

podobnie dla y :

$$y = \cdots = 10^{n/2} y_L + y_R$$

Wymnażamy:

$$x \cdot y = (10^{n/2} x_L + x_R) \cdot (10^{n/2} y_L + y_R) = 10^n x_L y_L + 10^{n/2} (x_L y_R + x_R y_L) + x_R y_R$$

Złożoność czasowa algorytmu – ozn. $T(n)$.

- Podział x, y na x_L, x_R, y_L, y_R zajmie $\mathcal{O}(n)$ czasu.
- Dodanie liczb n -cyfrowych zajmie $\mathcal{O}(n)$ czasu.
- Mnożenie przez 10^n jest równoważne dopisaniu n zer więc też $\mathcal{O}(n)$.

$$T(n) = 4 \cdot T(n/2) + \Theta(n)$$

Twierdzenie 1.2. *CLRS - Uniwersalne Twierdzenie o Rekurencji* Niech $a \geq 1$, $b > 1$ będą stałymi, $f(n)$ funkcją i $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & f(n) = \mathcal{O}(n^{\log_b a - \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ \Theta(n^{\log_b a} \log n) & f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \begin{array}{l} f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ a \cdot f(n/b) \leq c \cdot f(n) \text{ dla pewnej stałej } c < 1 \text{ dla dużych } n \end{array} \end{cases}$$

Lemat 1.3. Niech $a \geq 1$, $b > 1$ będą stałymi i niech $f(n)$ będzie nieujemną funkcją zdefiniowaną dla potęg b . Jeśli

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n = b^i, i \geq 1 \end{cases}$$

to

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Dowód.

$$a^{\log_b n} = b^{\log_b n \cdot \log_b a} = b^{\log_b a \cdot \log_b n} = (b^{\log_b a})^{\log_b n} = n^{\log_b a} \quad (1)$$

$$\begin{aligned}
 T(n) &= f(n) + a \cdot T\left(\frac{n}{b}\right) = f(n) + a \left(f\left(\frac{n}{b}\right) + aT\left(\frac{n}{b^2}\right) \right) \\
 &= f(n) + af\left(\frac{n}{b}\right) + a^2T\left(\frac{n}{b^2}\right) = f(n) + af\left(\frac{n}{b}\right) + a^2 \left(f\left(\frac{n}{b^2}\right) + aT\left(\frac{n}{b^3}\right) \right) \\
 &= f(n) + af\left(\frac{n}{b}\right) + a^2f\left(\frac{n}{b^2}\right) + \dots + a^{\log_b n - 1} f\left(\frac{n}{b^{\log_b n}}\right) + a^{\log_b n} T(1) \\
 &\stackrel{1}{=} \dots + a^{\log_b n} T(1) = \dots + n^{\log_b a} T(1) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)
 \end{aligned}$$

□

Przeprowadzimy teraz dowód uniwersalnego twierdzenia o rekurencji (1.2). Ograniczymy się do przypadku, gdy n jest potęgą b . Pozostały przypadek jest bardziej techniczny.

Dowód. Rozważmy wszystkie przypadki:

1.

$$f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$$

Wtedy

$$f\left(\frac{n}{b^j}\right) = \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right)$$

$$\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) = \sum_{j=0}^{\log_b n - 1} a^j \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right) = \mathcal{O}\left(\underbrace{\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}}_{\blacktriangle}\right)$$

$$\begin{aligned}
 \blacktriangle &= \sum_{j=0}^{\log_b n - 1} a^j \frac{n^{\log_b a - \epsilon}}{b^{j(\log_b a - \epsilon)}} = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} a^j \frac{b^{j\epsilon}}{b^{j \log_b a}} \\
 &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} \left(\underbrace{\frac{a \cdot b^\epsilon}{b^{\log_b a}}}_{=a} \right)^j = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} b^{\epsilon j} \quad (\text{wzór na ciąg geometryczny}) \\
 &= n^{\log_b a - \epsilon} \frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1} = n^{\log_b a - \epsilon} \frac{n^\epsilon - 1}{\underbrace{b^\epsilon - 1}_{\text{stała}}} \\
 &= n^{\log_b a - \epsilon} \mathcal{O}(n^\epsilon - 1) = \mathcal{O}(n^{\log_b a - \epsilon} \cdot n^\epsilon - 1) = \mathcal{O}(n^{\log_b a})
 \end{aligned}$$

Z lematu:

$$T(n) = \sum_{j=0}^{\log_b n - 1} a^j T\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a}) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a})$$

2.

$$f(n) = \Theta(n^{\log_b a})$$

Wtedy:

$$f\left(\frac{n}{b^j}\right) = \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right)$$

$$\begin{aligned} \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n-1} a^j \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) = \Theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\ &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} a^j \frac{1}{b^{j \log_b a}}\right) = \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} a^j \frac{1}{\underbrace{b^{\log_b a^j}}_{=a^j}}\right) \\ &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} 1\right) = \Theta(n^{\log_b a} \log_b n) \end{aligned}$$

Z lematu:

$$T(n) = \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a} \log_b n) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a} \log_b n)$$

3.

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

oraz

$$a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$$

dla jakiegoś $c < 1$ i odpowiednio dużych n .

Z założenia:

$$f\left(\frac{n}{b}\right) \leq \frac{c}{a} f(n) \quad \text{więc} \quad f\left(\frac{n}{b^j}\right) \leq \left(\frac{c}{a}\right)^j f(n) = \frac{c^j}{a^j} f(n)$$

$$\begin{aligned} \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) &\leq \sum_{j=0}^{\log_b n-1} a^j \frac{c^j}{a^j} f(n) + \mathcal{O}(1) \leq f(n) \sum_{j=0}^{\log_b n-1} c^j + \mathcal{O}(1) \\ &\leq f(n) \sum_{j=0}^{\infty} c^j + \mathcal{O}(1) \leq f(n) \frac{1}{1-c} + \mathcal{O}(1) = \mathcal{O}(f(n)) \end{aligned}$$

Z drugiej strony:

$$\sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) = a^0 f\left(\frac{n}{b^0}\right) + \underbrace{\sum_{j=1}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right)}_{\geq 0} = \Omega(f(n))$$

Z lematu:

$$T(n) = \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \Theta(f(n)) + \Theta(n^{\log_b a}) = \Theta(f(n))$$

□

Wniosek 1.4. Niech $a \geq 1$, $b > 1$ będą stałymi, a $f(n)$ funkcją taką, że $f(n) = \Theta(n^k)$. Niech $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & a > b^k \Leftrightarrow \log_b a > k \\ \Theta(n^{\log_b a} \log n) & a = b^k \Leftrightarrow \log_b a = k \\ \Theta(n^k) & a < b^k \Leftrightarrow \log_b a < k \end{cases}$$